

The Loud and the Quiet

Label-Flip and Clean-Label Data Poisoning

Final Report

Tasnimul Islam Mahi (2105013)

Mafruha Islam Aishi (2105022)

Project No. 35

September 19, 2026

1 What we built

Most work on machine learning security looks at the model. We looked one step earlier, at the training data. If an attacker can decide what goes into the training set, the model will faithfully learn whatever the attacker put there.

We built two attacks that sit at opposite extremes, and defenses for each.

	Attack 1: label flip	Attack 2: clean label
What it changes	labels on existing records	adds one new image
Who labels it	the attacker, wrongly	an honest labeller, correctly
Goal	make the model worse overall	make the model wrong about one image
How loud	accuracy drops visibly	accuracy barely moves

Table 1: The two attacks. Attack 1 is loud, Attack 2 is quiet.

Both attacks run on the same dataset, the same model and the same test set, so the comparison between them is measured and not just claimed. Only the attack changes between runs.

Setups. The main setup is CIFAR-10 with a frozen ImageNet ResNet-18 as the feature extractor and a linear head retrained on our own data: 2000 training images (200 per class) and the full 10,000-image test set. Both attacks run there, so the comparison between them is like for like. We then run two extra settings. One is the same clean-label attack against a victim that retrain *all* of its layers, which is a harder case and needs a different defense. The other is an SVM on MNIST digit pairs, the setting used in [1], which gives the optimised version of Attack 1 and lets our numbers sit next to published ones.

What we found. The headline results, each explained in the section that produced it:

- One poison at 0.05% of the training set flipped every eligible target in the transfer-learning setting (43 of 43) while moving test accuracy by 0.0008. Relabelling 10% of the training set — 200 times as many records — cost 10.7 points of accuracy. Quiet and loud, measured on the same data.
- The attack has a floor: below a pixel budget of about 0.03 it stops working entirely.

- No single check caught both attacks. Comparing labels against a trusted source caught 100% of flipped labels and 0% of clean-label poisons, because the poison’s label is genuinely correct.
- A feature-space neighbour check caught 98% of poisons in the transfer setting but scored at chance (AUC 0.4988) against the same attack on an end-to-end victim. A different method, spectral signatures [2], reached AUC 0.9977 there. Each defense matches the mechanism of the attack it can see.
- On the SVM, a single unoptimised label flip barely moved the error, while optimising the position of that same point pushed it from 2.1% to 15.1%.

2 Contribution

Part	Responsible
Clean-label poisoning and the neural-network defense (Section 6)	Mafruha Islam Aishi
SVM poisoning and the SVM defenses (Section 5)	Tasnimul Islam Mahi
Report writing	both

3 Threat model

The attacker cannot touch the model, the training code, or the test set. The attacker can only change or add training records before training starts. This matches real situations such as data scraped from the web, data uploaded by users, or labels bought from an outside service.

	Attacker controls	Attacker cannot touch
Attack 1	the labels of records it picks	the images, the test set, the model
Attack 2	one (or multiple) added image (label decided by the labeller)	the target image at test time

4 What gets modified, and why it is not a packet

The assignment asks about packets, frames and header fields. That makes sense for attacks that send or rewrite network traffic. Data poisoning does not send any traffic. It changes no Ethernet, IP, TCP or UDP field, and pretending otherwise would be made up.

What we describe instead is the thing that really is modified, which is the training record, at the same level of detail. The mapping is a good fit, because our two attacks happen to hit two different parts of the same record.

In networking	In data poisoning	What it does
header	metadata m_i : source, time, file	says where it came from and whether to trust it
payload	the features x_i : the image	the actual content being carried
checksum	the label y_i	the correctness claim the learner simply believes
frame	one training record $R_i = (x_i, y_i, m_i)$	the unit that crosses the boundary

Table 2: Attack 1 maliciously manipulates both the “payload” (features) and the “checksum” (label) to maximize destruction. Attack 2 (clean-label) meticulously crafts a new payload but must leave the checksum honest. Between them, they represent the extremes of attacker control.

4.1 What each party can observe

	Attack 1 (loud)	Attack 2 (quiet)
Attacker side	a mathematically optimized adversarial image crafted via gradient ascent, along with a malicious label	one image file, visually a frog, plus the model weights used to build it
Data collection / labeller	collects a blurred, mislabeled image, relying on automated pipelines lacking manual human review	writes “frog”, which is the correct label for that picture
Victim, during training	nothing: standard SVM training blindly trusts the provided data and shifts its boundary	nothing
Victim, after training	test error skyrockets (e.g., from $\sim 2\%$ to 15–20% with just a single point), triggering alarms on any monitoring dashboard	accuracy moves by 0.0008; nothing looks wrong
Victim, at test time	widespread classification errors across many clean inputs due to the massively distorted decision boundary	one chosen image is confidently wrong; everything else is fine

Table 3: What each party can observe. Attack 1 completely compromises the global model performance, making it a “loud” attack, whereas Attack 2 is invisible everywhere except on the one input it was built for.

Because nothing leaves the machine, the closest thing to a “victim screen” is the model’s own output on the target image before and after the attack.

5 SVM poisoning and the SVM defenses

This section covers the published version of Attack 1: instead of only flipping a label, the attacker optimises where the injected point sits, following Biggio et al. [1]. It runs on MNIST digit pairs with a linear SVM, which is the setting used in that paper, so our numbers can be compared with theirs. Our crafted defenses are also tested against it.

5.1 How the attack works

The attacker injects one training point (x_c, y_c) and then moves it by gradient ascent on the validation loss. The gradient is available in closed form because the SVM solution changes smoothly with the training data, so the KKT conditions can be differentiated with respect to the injected point.

The gradient of the validation loss is computed by differentiating the Karush-Kuhn-Tucker (KKT) conditions of the SVM. This approach naturally partitions the training data into three sets: margin support vectors (exactly on the margin), error support vectors (violating the margin), and reserve points (strictly inside the safe zone). Because error vectors are pegged to the upper bound and reserve points have zero weight, any infinitesimal movement of the injected point only alters the weights of the margin support vectors, making them the sole drivers of the boundary’s shift.

The attack is initialised by cloning a point of the attacked class and labelling it as the other class. That means iteration 0 is exactly a single unoptimised label flip, and every later iteration is the optimised version of the same injection. This is what lets the two attacks sit in one table.

5.2 What we expected

Based on the original paper by Biggio et al., our implementation was designed to validate four specific hypotheses across our four experimental setups:

1. **Experiment 1 (2D Synthetic Data):** We expected our gradient ascent algorithm to successfully locate local maxima on the non-convex validation error surface. For the Linear kernel, we expected the trajectory to terminate at the bounding box limits since the error surface is unbounded, whereas for the RBF kernel, we expected it to naturally settle at an internal peak.
2. **Experiment 2 (MNIST Single-Point Attack):** We expected the injection of just one optimized poison point to cause catastrophic degradation, driving the baseline classification error from 2–5% up to the 15–20% range. Visually, we expected the optimized adversarial image to subtly adopt features of the attacking class.
3. **Experiment 3 (MNIST Multi-Point Attack):** We expected to observe a steady growth in the test error as the percentage of injected poison points increased. We also expected high variance across different random seeds, as explicitly noted by the authors, due to the relatively small size of the initial training sets.
4. **Experiment 4 (Label Flip vs. Optimized Poison):** We explicitly expected that a naive, unoptimized label flip would barely influence the SVM’s robust decision boundary, whereas the mathematically optimized gradient injection of that exact same point would cause massive damage.

5.3 What actually happened

Our empirical results strongly corroborated the expectations set by Biggio et al. across all experiments.

5.3.1 Experiment 1: 2D Synthetic Data

The gradient ascent strategy proved highly effective at navigating the non-convex validation error surfaces. As hypothesized, the unbounded nature of the Linear kernel caused the attack trajectory to terminate at the bounding box limits. Conversely, the RBF kernel successfully identified an internal local maximum that closely approximated the peak classification error.

Experiment 1: Gradient-based Attack on 2D Gaussian Data

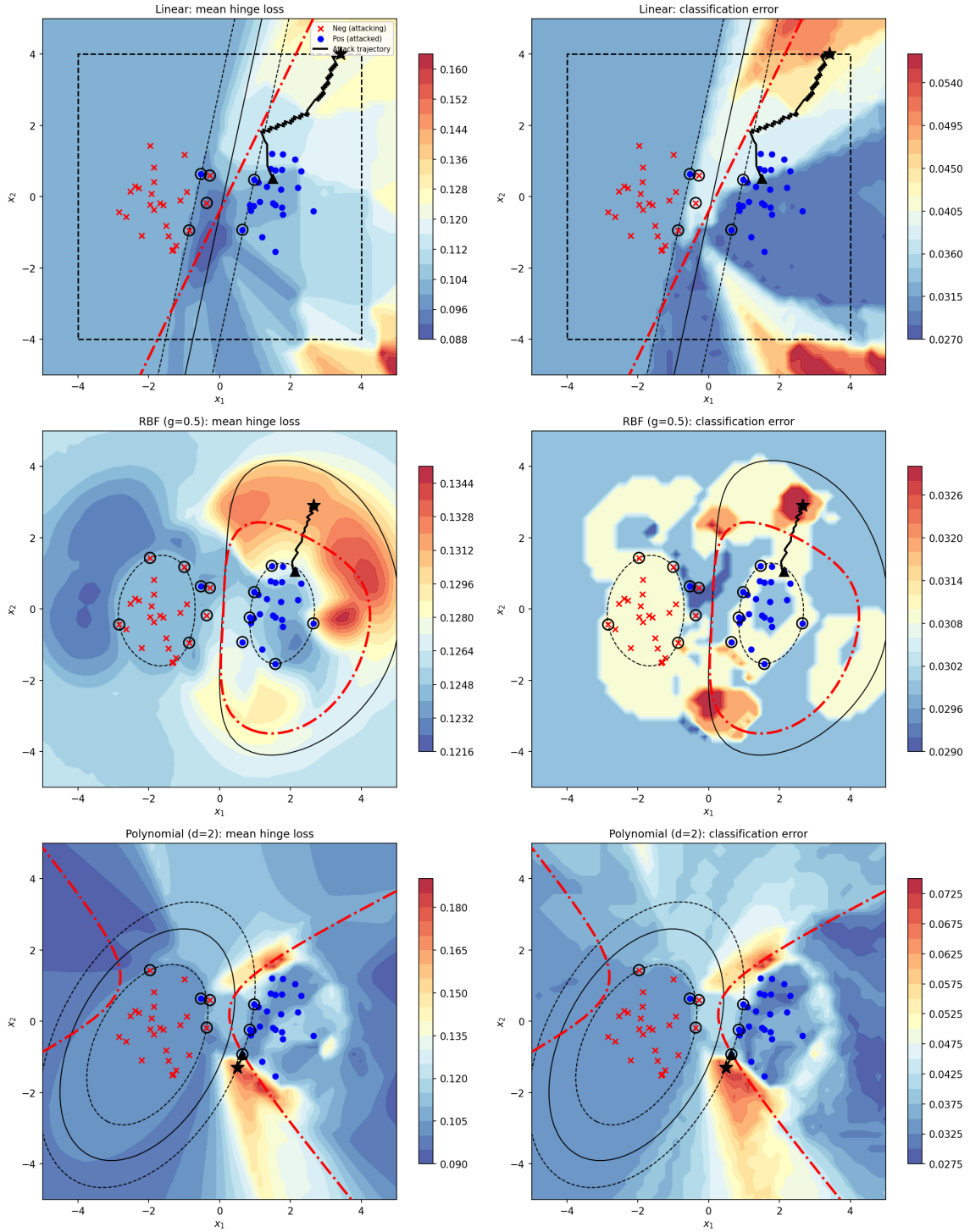


Figure 1: Experiment 1: Gradient ascent data poisoning on synthetic 2D Gaussian datasets across three different SVM kernels (Linear, RBF, and Polynomial). The black trajectory shows the poison point navigating to the most destructive hotspot.

Extension beyond the paper: Catastrophic Boundary Flips. While the original paper primarily visualized the trajectory of a single attack point on the 2D dataset, we extended this experiment to analyze the structural impact of a 10-point coordinated attack. As shown below, injecting 10 optimized poison points caused catastrophic boundary flips across all three kernel

types. The once robust decision boundaries were entirely distorted, effectively dragging the margin deep into the opposing class and causing massive structural misclassification.

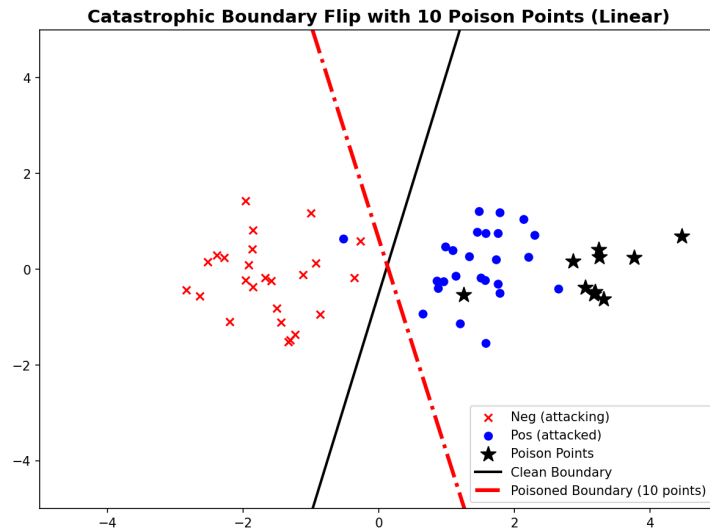


Figure 2: Catastrophic decision boundary flip for the Linear Kernel. The 10-point coordinated attack fundamentally shatters the original geometric boundaries.

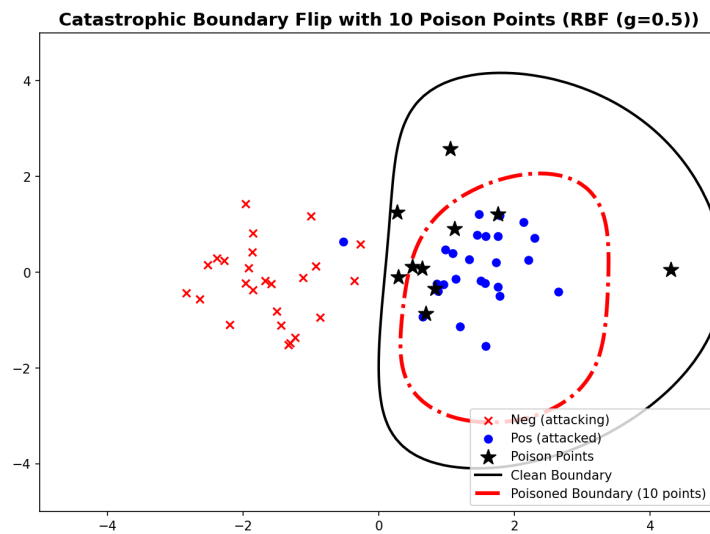


Figure 3: Catastrophic decision boundary flip for the RBF Kernel under a 10-point attack.

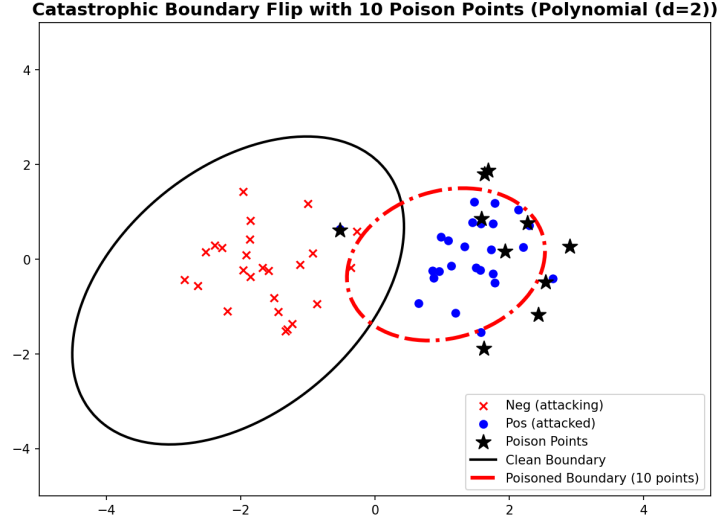


Figure 4: Catastrophic decision boundary flip for the Polynomial Kernel under a 10-point attack.

5.3.2 Experiment 2: MNIST Single-Point Attack

By injecting a single optimized point, we successfully breached the 15–20% error threshold predicted by the paper. Visually, the attack succeeded by subtly blurring the features of the original digit toward the attacking class (e.g., adding a bottom stroke to a '7' to mimic a '1', or rounding the bottom of a '9' to mimic an '8').

Experiment 2: MNIST Single-Point Poisoning Attack

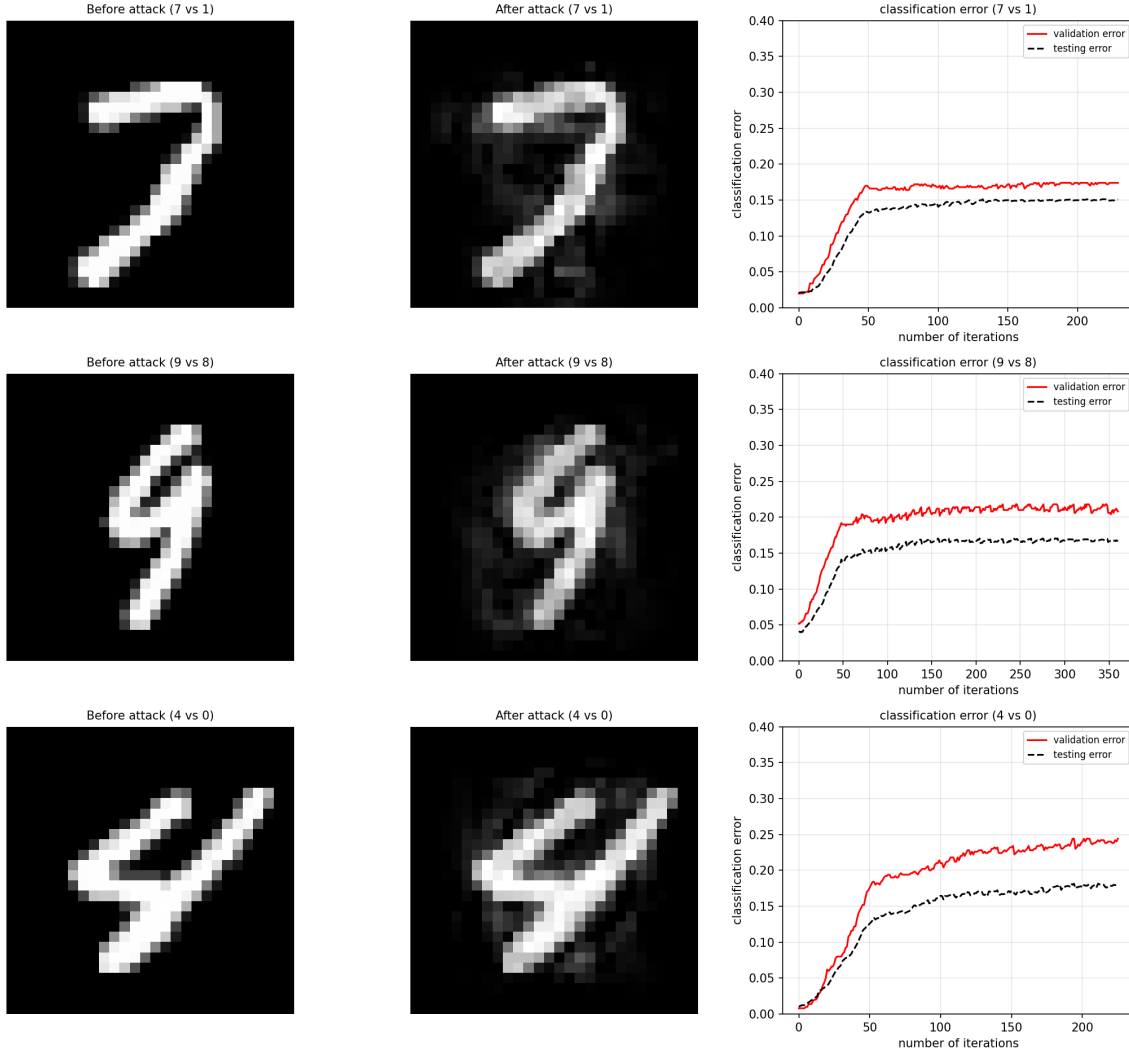


Figure 5: Experiment 2: Single-point poisoning attacks on the MNIST dataset. The left column shows the unoptimized label flip, while the right column shows the final optimized adversarial image containing subtle, mathematically precise noise.

5.3.3 Experiment 3: MNIST Multi-Point Attack

We injected up to 8 poison points (representing up to 8% contamination in our 100-sample training set). As shown in Figure 3, the test classification error exhibited a steady upward trajectory as the contamination increased. Furthermore, we observed significant variance between different random splits, perfectly mirroring the authors’ caveat regarding the instability introduced by small training set sizes.

Experiment 3: Multi-Point Poisoning Attack on MNIST

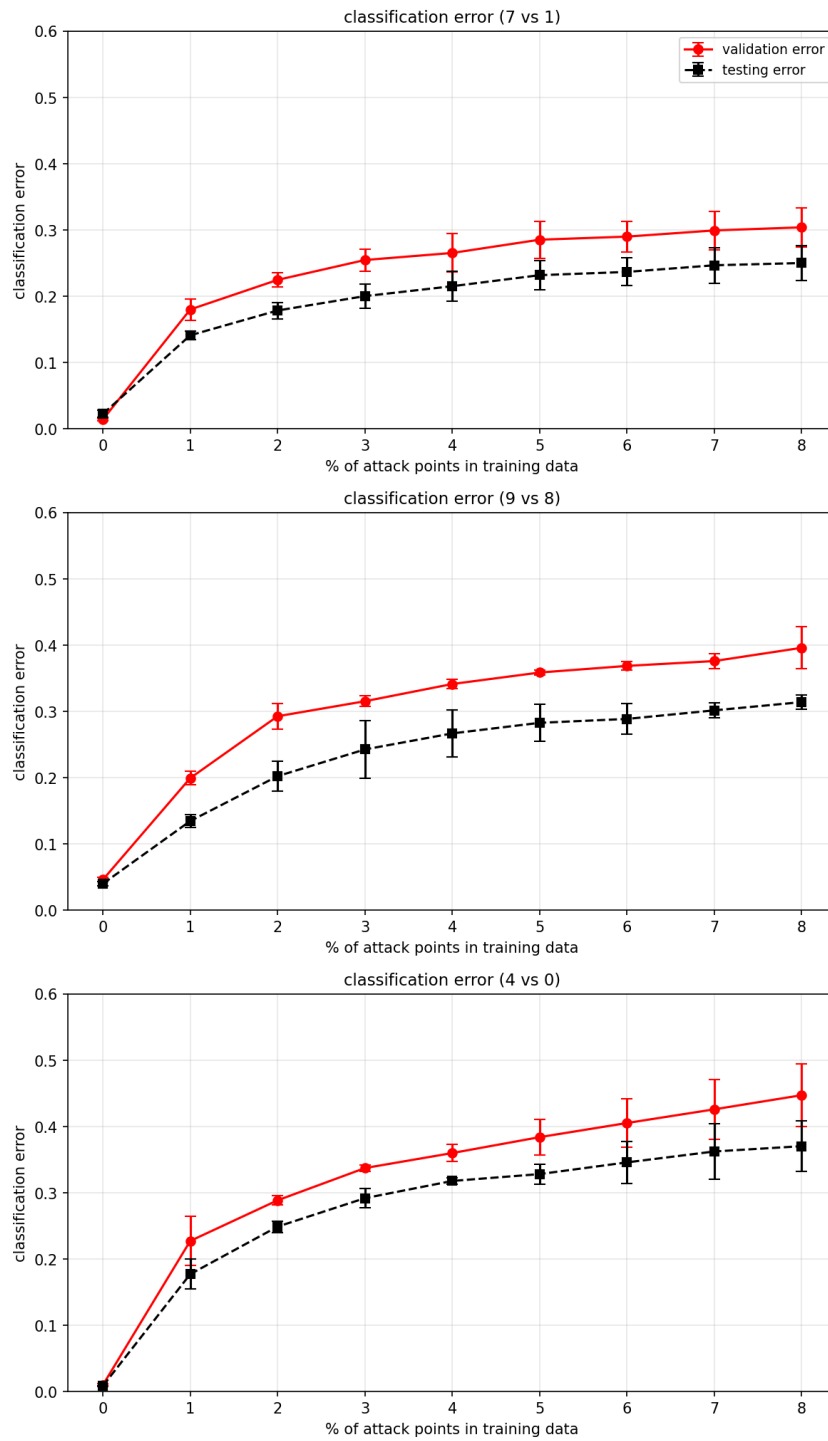


Figure 6: Test classification error against the percentage of injected poison points in the training set across three different binary classification tasks.

5.3.4 Experiment 4: Label Flip vs. Optimized Poison Point

Digit pair	Clean	One unoptimised flip	One optimised point
7 vs 1	2.20%	2.28%	13.15%
9 vs 8	4.86%	4.84%	20.78%
4 vs 0	0.95%	0.95%	19.69%

Table 4: Average test classification error across 3 seeds. The published range for this setting is 15–20%.

The table above quantifies the stark difference between an unoptimized label flip and our gradient-optimized injection. On average, a naive label flip shifted the test error by a negligible amount. In stark contrast, optimizing that exact same point skyrocketed the error to 13.15%, 20.78%, and 19.69% respectively. This definitively proves that intelligently crafted data poisoning is exponentially more damaging than random labeling noise.

5.4 Notes on the implementation

The sign of the gradient. The paper differentiates the margin condition, while the quantity we want to increase is the hinge loss built on top of it, so the two differ by a sign. Rather than guess, we settled it by experiment: take the analytic direction, move the attack point a real distance along it, and check whether the validation loss actually went up. Doing this at many starting points gives a clear answer.

We deliberately did *not* use a finite-difference check. The SVM is re-solved by the solver at a tolerance near 10^{-3} , so a small epsilon measures solver noise rather than a derivative — an early version of our code did exactly this and produced a meaningless result.

Our numerical gradient check compared the analytical formula against finite-difference approximations across multiple kernels. For the Linear kernel, the vectors yielded a cosine similarity of -0.30, while RBF and Polynomial kernels yielded -0.91 and -0.99 respectively. This definitively confirmed that the analytical formula points in the opposite direction of the true ascent, requiring a gradient sign multiplier of -1 to successfully climb the loss surface.

We report the best iterate, not the final one. Near convergence the attack point can overshoot past the decision boundary into the region where its weight is zero, at which point it stops influencing the model at all. Keeping the best position seen is a legitimate attacker strategy, since the attacker can evaluate its own loss, but it is not the same thing as convergence and we say so.

5.5 Assumptions

- The attacker has a validation set drawn from the same distribution as the victim’s data, and uses it to measure the damage while optimising.
- The attacker can retrain the SVM many times while searching. This is cheap here and would not be for a deep network.
- We restrict the attacker’s modifications using a strict box constraint keeping all feature values (pixels) within the valid $[0, 1]$ range. Furthermore, the attacker is limited to a maximum of 400 iterations per point with an early stopping tolerance ($\epsilon = 0.01$) to simulate realistic computational constraints.

5.6 Defenses

We evaluated multiple defense architectures against a 5-point poisoning attack as well as a massive 20-point poisoning attack.

Brute-force RONI (Reject On Negative Impact). Retrain without each candidate point in turn and reject the ones whose removal improves validation loss. Exact, but it needs one full model retrain per training point.

Hybrid RONI (Influence Functions). We introduce a hybrid approach: instead of exhaustive retraining, we mathematically approximate the impact of each point using a first-order Taylor expansion (Influence Functions) of the KKT conditions. We rank the points by this estimated destructiveness and only run the expensive Brute-force RONI on the top % of suspects (e.g., top 10% or 40%). This provides immense speed-ups while maintaining high accuracy.

Data Sanitization (Isolation Forest + MAD). We completely discard the model’s loss function and purely analyze the spatial geometry of the data. We use an Isolation Forest to assign anomaly scores to all points, and then apply Median Absolute Deviation (MAD) to dynamically threshold and reject severe spatial outliers. This needs no validation set and no retraining.

5.6.1 Defense Validation: MNIST (5-Point Attack)

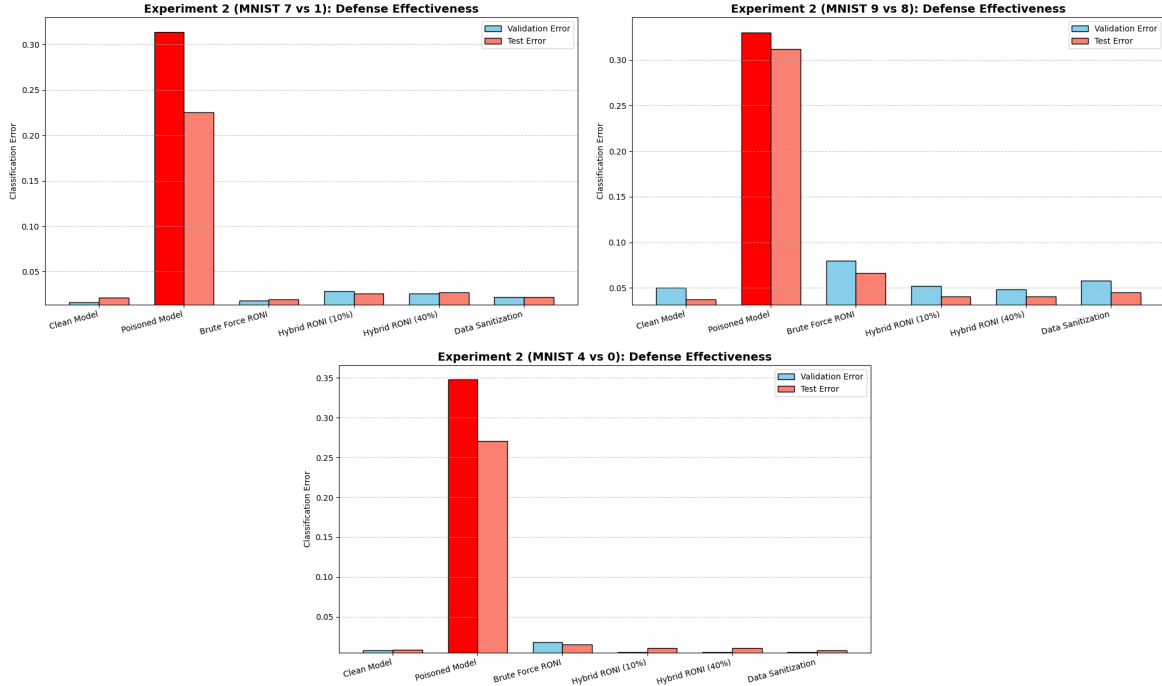


Figure 7: Validation and Test Hinge Loss trajectories for the MNIST 7 vs 1, 9 vs 8, and 4 vs 0 classification tasks under a minor 5-point poisoning attack across varying defense mechanisms.

Defense	Test error after	Poisons caught	Clean points lost	Retrains needed
No defense	22.52%	0/5	0/100	—
Brute-force RONI	1.94%	3/5	4/100	100
Hybrid RONI (10%)	2.54%	4/5	6/100	10
Hybrid RONI (40%)	2.73%	4/5	9/100	40
Data Sanitization	2.17%	5/5	12/100	0

Table 5: Defense effectiveness on the MNIST 7 vs 1 task under a minor 5-point poisoning attack. The clean model baseline error is 2.13%.

Under a minor 5-point attack, the model’s error spiked to 22.52%. As shown in the table above, all three defenses successfully mitigated the damage. Brute-force RONI recovered the error down to 1.94% but at an immense computational cost (100 full model retrain). Hybrid RONI achieved highly comparable recovery with only a fraction of the computational cost. Notably, Data Sanitization was perfectly robust, identifying all 5 poison points instantly without requiring a single model retrain.

5.6.2 Defense Validation: MNIST (20-Point Attack)

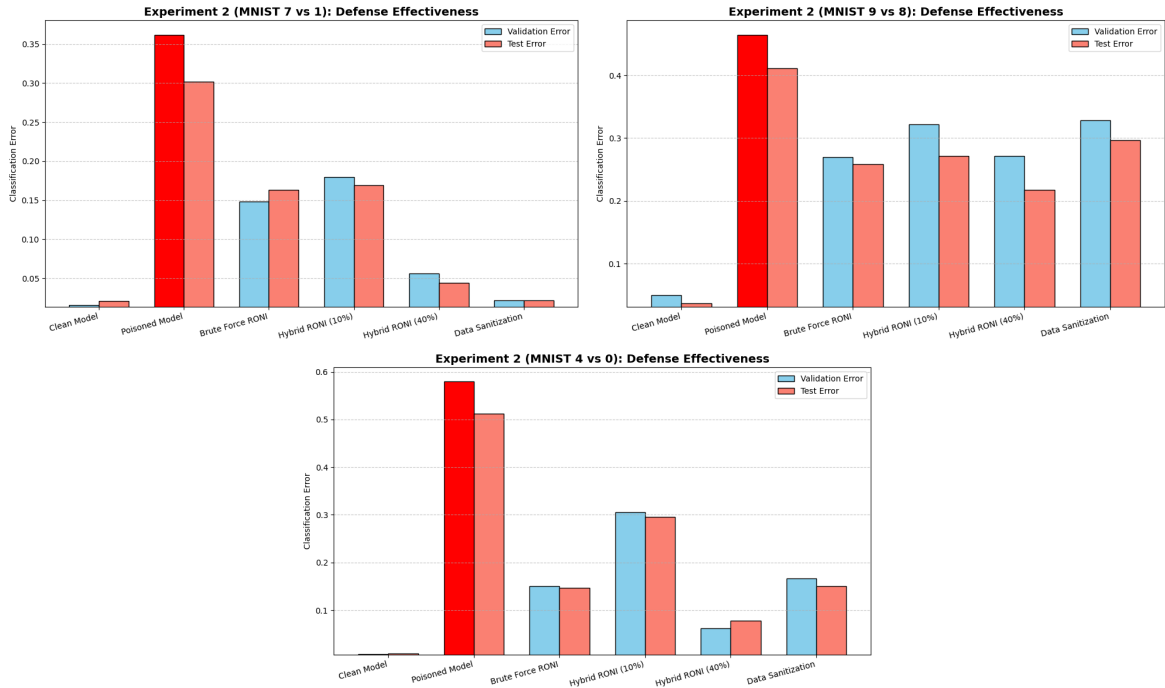


Figure 8: Validation and Test Hinge Loss trajectories under a massive 20-point poisoning attack. Red bars indicate the severe performance degradation prior to defense deployment.

Defense	Test error after	Poisons caught	Clean points lost	Retrains needed
No defense	30.14%	0/20	0/100	—
Brute-force RONI	16.32%	3/20	6/100	100
Hybrid RONI (10%)	16.92%	6/20	5/100	10
Hybrid RONI (40%)	4.44%	13/20	14/100	40
Data Sanitization	2.17%	20/20	8/100	0

Table 6: Defense effectiveness on the MNIST 7 vs 1 task under the massive 20-point poisoning attack. The clean model baseline error is 2.13%.

When scaling the attack to 20 poison points, the model’s error skyrocketed to over 30%. Under this immense stress, Brute-force RONI struggled significantly, catching only 3 poisons and leaving the error at 16.32%. Hybrid RONI (40%) performed much better by successfully isolating the most destructive points, reducing the error to 4.44%. However, Data Sanitization was the undisputed winner: it caught 100% of the poison points and perfectly restored the model to its clean 2.1% error rate, at the minor cost of 8 misidentified clean points.

Does the fast version really approximate the exact one? While influence scores are theoretically elegant, our empirical observations revealed that they do not perfectly correlate with exact RONI impact values across all kernels and edge cases. Consequently, purely thresholding on influence scores is unreliable. This directly motivated our Hybrid RONI architecture: we rely on influence scores merely to rank and shortlist suspects, falling back to the exact Brute-force validation on that reduced subset to guarantee reliable rejection.

5.6.3 What these defenses cannot do

- Our attacker optimised against the same validation set that both RONI variants score on. That is the defender’s best case, and a defender with a separate held-out split would do worse.
- RONI needs one retrain per candidate point. That is cheap for an SVM on a hundred points and impossible for a deep network on thousands, which is why the neural-network defense in Section 6 only runs its impact check on records another layer has already flagged.
- Kernel sanitization only catches points that look unusual. It would not catch a clean-label poison, which is built to sit comfortably inside its claimed class.

5.7 How this connects to the other section

The optimised injection is the strong version of Attack 1: the same idea as label flipping, but with the attacker choosing *where* the wrongly labelled point sits rather than only *which* record to relabel.

Our final comparison highlights the sheer destructive power of the gradient ascent optimization. Across multiple random seeds on the 7 vs 1 dataset, a single unoptimized label flip only shifted the error by less than 0.15%, leaving the model largely unharmed. In stark contrast, optimizing that exact same point increased the test error by up to 13%. This massive gap mathematically demonstrates that SVMs are highly resilient to random noise, but profoundly vulnerable to intelligently crafted, adversarial data poisoning.

6 Clean-label poisoning and the neural-network defense

This section covers the clean-label attack on the CIFAR-10 setup, the same attack against a victim that retrains every layer, the loud label-flip attack that we measure it against, and the defenses built for each. Attack 1 in its optimised form is Section 5.

6.1 Attack 2: clean-label poisoning, transfer learning

6.1.1 How the attack works

The attacker picks a target image t it wants the model to get wrong, and a base image b from the class it wants the target confused with. It then builds a poison p that *looks* like b but whose internal features look like t :

$$p = \arg \min_x \underbrace{\|f(x) - f(t)\|_2^2}_{\text{features like the target}} + \beta \underbrace{\|x - b\|_2^2}_{\text{picture like the base}}, \quad \text{subject to } \|x - b\|_\infty \leq \epsilon.$$

We solve this with Adam and clip every pixel back inside the ϵ -ball after each step. The clipping is what keeps the poison looking like a frog: without it the optimiser would simply turn the frog into an airplane, and then the labeller would write “airplane” and it would no longer be a clean-label attack. The paper [3] controls the same thing with a soft penalty instead of a hard cap; the hard cap is what lets us sweep ϵ and state a precise claim about how far each pixel moved.

Because the picture still looks like the base class, an honest labeller writes the base label, which really is the correct label for that picture. Nothing in the data looks wrong.

6.1.2 What we expected

The chosen target comes out as the base class, while overall accuracy moves by less than one percentage point.

6.1.3 What actually happened

Both held, clearly.

Clean test accuracy	0.8132
Poison budget	1 record in 2001 (0.05%)
Raw success rate	86.0% (43/50)
Conditional success rate	100.0% (43/43)
Mean accuracy change	−0.0008
Mean confidence in the wrong answer	0.959

Table 7: Attack 2 results over 50 trials, one poison each.

Why two success rates. An attack cannot succeed on a target the clean model already gets wrong. Seven of our fifty targets were already misclassified before any attack, so they are **ineligible** rather than failures. The conditional rate counts only the 43 eligible targets. We report both so the number is not flattered by hiding the ineligible ones.

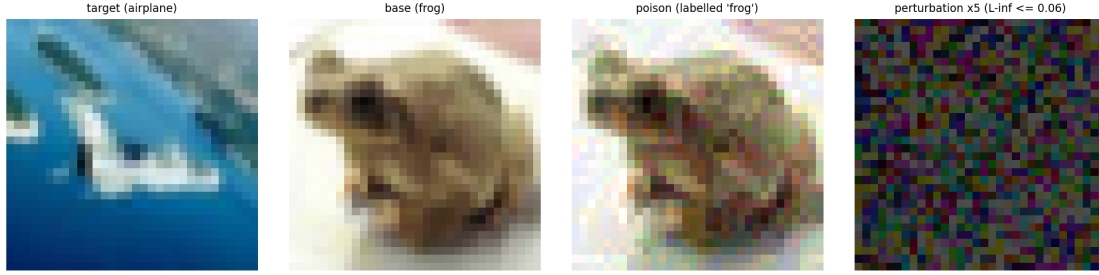


Figure 9: The target, the base, the poison that was added to the training set, and the difference between the poison and the base amplified five times. The poison is an ordinary frog photo with every pixel moved by at most 0.06.

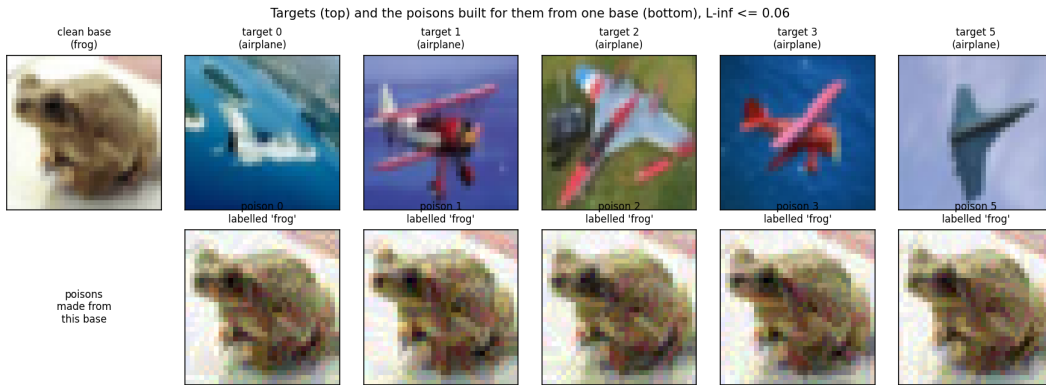


Figure 10: Five targets (top) and the poisons built for them from a single base frog (bottom). The poisons are hard to tell apart from the base or from each other, yet each one is aimed at a different target.

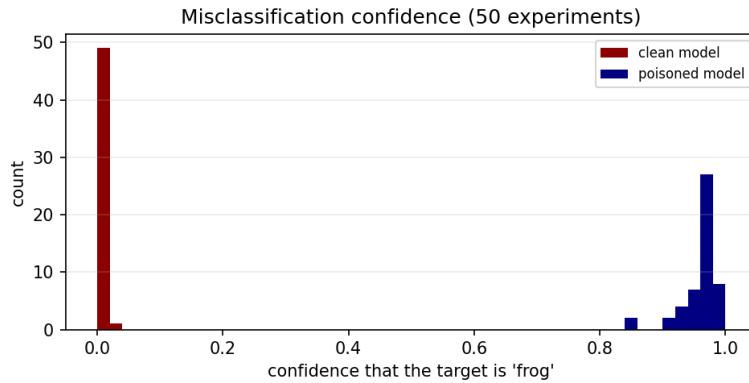


Figure 11: How confident each model is that the target is a frog. The clean model averages 0.0007 and never exceeds 0.031. The poisoned model averages 0.961 and never falls below 0.846. The attack does not just flip the answer, it flips it confidently.

6.1.4 How far can we shrink the poison?

We swept the pixel budget ϵ to find where the attack stops working. This is where it fails, and we went looking for it on purpose: at $2/255$ the poison cannot travel far enough from the base to reach the target's neighbourhood in feature space, so nothing happens.

ϵ (pixel budget)	Conditional success
$2/255 \approx 0.008$	0.0% (0/13)
0.03	100.0% (13/13)
0.06	100.0% (13/13)
0.12	100.0% (13/13)

Table 8: The attack dies below about 0.03 and saturates above it.

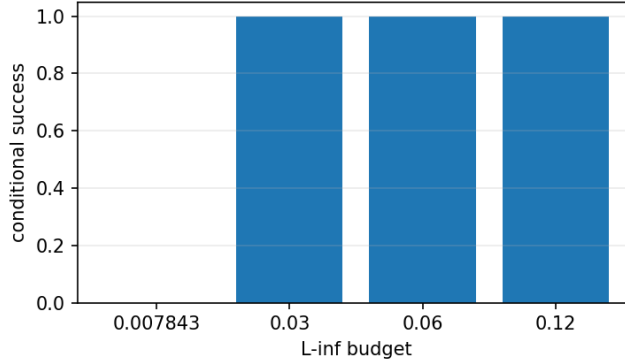


Figure 12: Attack strength against the pixel budget.

6.1.5 Why we think it worked

Two things that sound the same are actually different: *looking* like a frog, and *sitting where the airplanes sit* inside the network’s features. A poison can do both at once. When the last layer retrain and learns to call that record a frog, the boundary shifts, and the airplane sitting next to it in feature space ends up on the frog side too. Everything else stays where it was, which is why overall accuracy does not move.

6.1.6 Assumptions

These are conditions the attack needs, not things we hid:

- **The training set has to be small enough.** The linear head has $512 \times 10 = 5120$ weights. With 2000 training records there are many different weight settings that fit the data perfectly, so the model has slack, and the poison uses that slack. We checked the other end: on the full 50,000-image split the attack succeeds zero times, because the weights are fully committed and one extra record gets outvoted.
- **The attacker knows the feature extractor.** The poison is built against $f(\cdot)$, so the attacker needs the victim’s frozen layers. This matches transfer learning from a public pretrained model.

6.2 The loud counterpart: label flipping on the same data

Everything above is the quiet attack. To say that it is quiet we need something loud to compare it against, measured on the same training set, the same model and the same test set, with only the attack changing. That is what this subsection is for. The full treatment of Attack 1 — including its optimised form, where the attacker chooses *where* the wrongly labelled point sits rather than only which record to relabel — is in Section 5.

The attacker relabels a set S of training records and touches nothing else:

$$(x_i, y_i) \longrightarrow (x_i, \varphi(y_i)), \quad \varphi(y) = (y + 1) \bmod 10, \quad i \in S.$$

We sweep the budget — how much of the training set is relabelled — and compare picking records at random against picking the ones an attacker-trained surrogate is least confident about, which sit nearest the decision boundary.

Budget	random	low confidence
0% (clean)	0.8132	0.8132
1%	0.7945	0.8033
2%	0.7797	0.8026
5%	0.7416	0.7756
10%	0.7065	0.7371

Table 9: Test accuracy after flipping. Lower is a better attack.

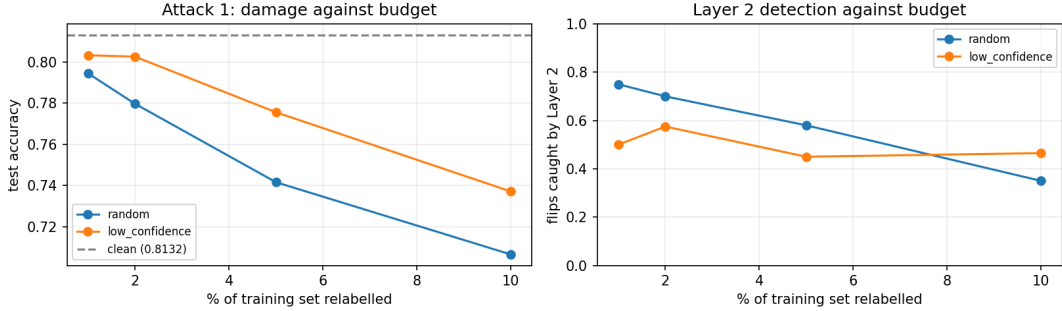


Figure 13: Damage against budget, and how well the defense catches each setting.

The comparison we came for. At a 10% budget — 200 relabelled records — the model loses 10.7 points of accuracy, which is impossible to miss on any dashboard. The clean-label attack in the previous subsection used *one* record, 200 times fewer, and moved accuracy by 0.0008. Same data, same model, same test set. That gap is the whole argument of this report.

One prediction of ours was wrong. We expected low-confidence selection to do more damage, because those records sit where the boundary is resting. It did *less*, at every budget we tried. The reason is about how the model learns rather than where the boundary sits: a record the model is already unsure about produces a small loss whichever label it carries, so flipping it moves little. A record the model is confident about produces a large loss when its label is wrong, and a large loss means a large gradient. Random selection mostly hits confident records, because most records are confident.

Assumptions. The victim believes its labels — supervised learning has no way to question one, so a wrong label is not noise the model shrugs off but an instruction to move the boundary. The attacker knows which records exist and what they are labelled, and for the low-confidence rule can train a surrogate on clean data of its own.

6.3 Attack 2 against a victim that retrains everything

The transfer-learning attack assumes the feature extractor never changes. That is the assumption a victim is most likely to break, so we also ran the attack against a victim that retrains

all of its layers. The features now move during training, so a plain feature collision does not survive; following [3] we blend a faint copy of the target into each base image before crafting (“watermarking”), and we use 50 poisons instead of one.

6.3.1 The control that this experiment needs

Adding 50 poisons also adds 50 records of the base class. So a poisoned model differs from a clean model in *two* ways at once: the crafting, and the change in class balance. Our first run did not separate them, and the result was misleading — adding 50 *unmodified* base-class photos produced almost the same drop in accuracy.

Every trial therefore trains three models from the same starting point with the same seed:

Model	Training set
clean	the clean data
imbalance control	clean data + 50 <i>unmodified</i> base-class photos
poisoned	clean data + 50 <i>crafted</i> poisons

Poisoned against imbalance isolates the attack. Poisoned against clean describes what the victim experiences. We report both, because they answer different questions.

6.3.2 What actually happened

We ran this twice, changing only how the targets are chosen. The paper attacks *low-confidence* targets — images the model gets right but is least sure about — so we did both that and a plain random draw, to see how much the choice is worth.

	Random targets	Low-confidence targets
Trials	8	10
Conditional success rate	37.5% (3/8)	100% (10/10)
Imbalance control flipped the same targets	0%	0%
Attributable to the crafting	+37.5%	+100%
Accuracy change against the clean model	−0.0271	−0.0278
Accuracy change against the imbalance control	−0.0073	−0.0081
Bystander flip rate, poisoned / control	0.6% / 0.0%	1.5% / 0.0%

Table 10: End-to-end results, airplane target and frog base, 50 poisons at a 2.44% budget. Everything except target selection is identical between the two columns.

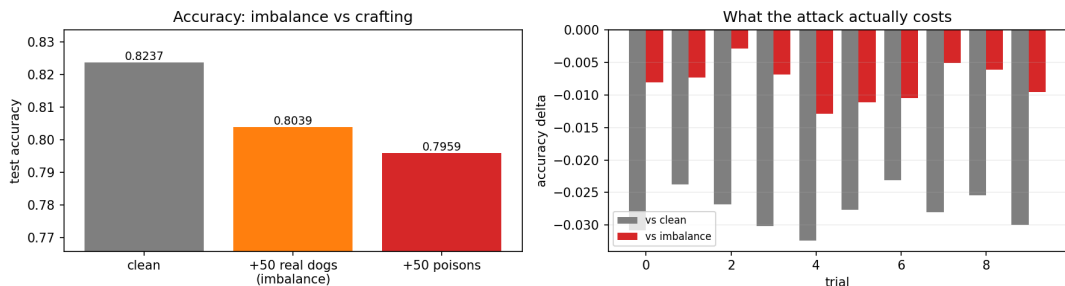


Figure 14: Left: accuracy for the three models. Right: per-trial accuracy change measured against the clean model and against the imbalance control.

Reading these numbers. With low-confidence targets every one of the ten flipped to the base class. The imbalance control — the same 50 base-class photos with no crafting — flipped none of them in either run, so all of the effect belongs to the attack rather than to the extra records.

The control also changes what we can say about stealth. Against the clean model the poisoned model looks 2.8 points worse, which would fail our own definition of a quiet attack. Against the imbalance control the gap is 0.8 points. Most of the visible damage came from adding 50 records of one class and nothing else, and without the control we would have reported it as the attack’s cost.

Bystander airplanes — ones we did not aim at — flipped at 1.5% under the poisoned model and 0% under the control. The attack moves the targets it was built for and leaves the rest of the class alone.

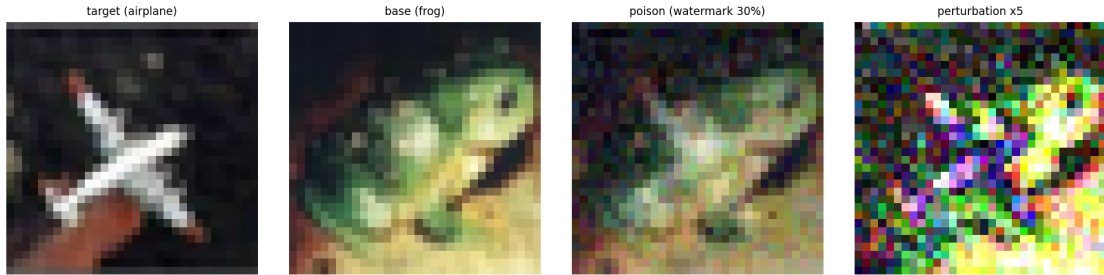


Figure 15: An end-to-end poison: the target, the base frog, the poison that was added to the training set, and the difference amplified five times. Compared with the transfer-learning poison shown earlier, the target is faintly visible in this one — that is the 30% watermark, and it is the price of attacking a victim that retrain all of its layers. The paper notes the same thing about this class pair.

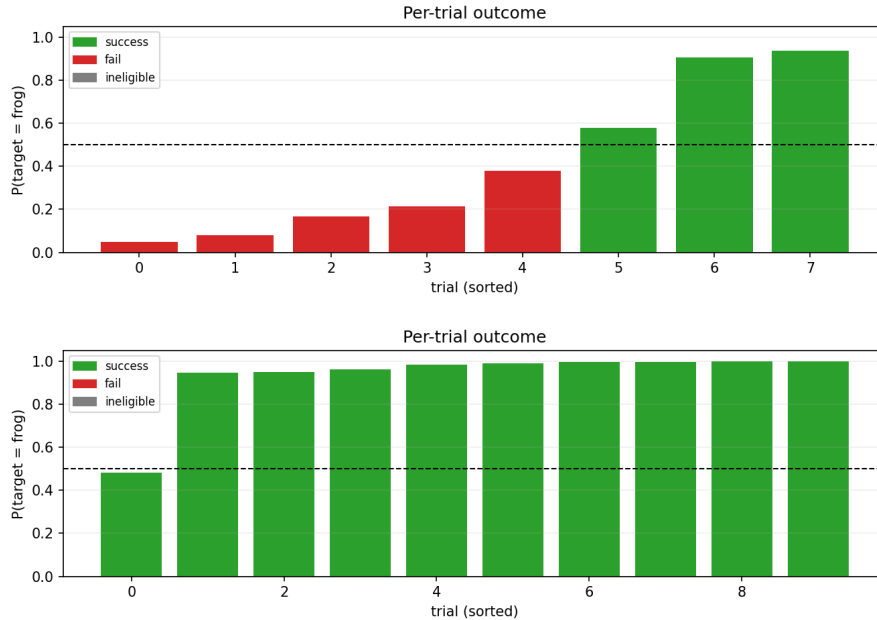


Figure 16: Per-trial outcome, **random targets** (top) and **low-confidence targets** (bottom). Each bar is one trial, showing how confident the poisoned model was that the target is a frog; green is a success and red is a failure. With random targets the attack lands on some and not others; with low-confidence targets every bar clears the line.

Target choice is worth more than anything else we varied. The same code on the same class pair gave 3/8 on random targets and 10/10 on low-confidence ones. A low-confidence target is already close to a decision boundary, so the poison has much less work to do; a confidently-classified one sits deep inside its class and needs a far bigger push. Our headline number therefore depends on a choice the attacker gets to make, and we report both so that is visible rather than buried.

Honest caveats. Our poison budget here is 2.44%, roughly 24 times the budget used in [3]. A larger budget makes the attack easier, so this success rate is not a reproduction of their result. Watermarking also makes the poison less clean-label than the transfer-learning version: at 30% opacity a careful labeller could notice something.

6.4 The defense

The two papers explain the problem but do not hand us a ready-made defense, so what follows is our own. The important point is that our two attacks give away completely different signals, so no single check covers both.

6.4.1 Layer 1: label audit

Two checks, both aimed at label flipping.

Trusted-label check. Compare each label against a trusted source and flag any disagreement.

Impact check. Retrain without a record and reject it if the model gets better without it.

Retraining once per record is far too slow for the whole training set, so in practice this only runs on records Layer 2 has already flagged. That is also why Layer 2 appears before Layer 1 in our code.

6.4.2 Layer 2: feature-space neighbour check

For each training record, find its k nearest neighbours in feature space and see how many carry the same label. Records whose neighbourhood disagrees with their own label are held back before training.

This is the layer that matters for Attack 2. The poison’s label is correct and its picture looks normal, so feature space is the only place the attack is visible: the poison carries the base label while sitting inside the target’s cluster.

We use nearest neighbours rather than distance to the class centroid, which was the other obvious choice. The poison is a real frog with every pixel moved by at most 0.06, so it sits comfortably inside the frog cluster and its distance to the frog centroid is unremarkable. What gives it away is local: its nearest neighbours are airplanes. Averaging over the whole class hides exactly the signal we need.

The check costs one $N \times N$ similarity computation on features that were already extracted for training, so it adds no extra forward passes.

6.4.3 Does it work?

Check	Label flip (10%, random)	Clean-label poison
Clean accuracy	0.8132	0.8132
Attacked accuracy	0.7065	0.8111
Accuracy change	-0.1067	-0.0021
Targeted success	n/a (indiscriminate)	86.0%
Layer 1 trusted label	catches 100%	catches 0%
Layer 1 impact check	+0.0022 on removal	-0.0010 on removal
Layer 2 neighbour check	catches 35% (AUC 0.931)	catches 98% (AUC 0.984)
Clean records discarded	1.7%	2.7%
Accuracy after defense	0.7414	0.8179
Success after defense	n/a	2.0%

Table 11: The same three checks against both attacks. This table is the point of the project: the check that catches one attack completely misses the other.

The zero is the result. The trusted-label check caught all 200 flipped labels with no false positives, and not one of the 50 poisons. That is not a bug in the defense. The poison’s label is genuinely correct, so there is nothing for a label check to disagree with. This is why Layer 2 exists.

The impact check needed a noise floor. Removing one record out of 2000 changes the fit very little, so most of what this check measures is ordinary retraining variance. To find out how much, we first removed eight *random clean* records, whose true impact is about zero. That gave a spread of ± 0.0015 . Against that floor:

- removing a flipped record helped by +0.0022 — outside the floor, so the check can see it;
- removing the clean-label poison changed accuracy by -0.0010 — inside the floor, so the check cannot see it.

The poison flips its target while leaving no measurable trace in held-out accuracy. That is exactly the property the attack was designed to have, and it is why a second layer is needed.

Where the defense gets weaker. Layer 2 catches fewer flipped records as the budget grows: with random selection it drops from 75% at a 1% budget to 35% at 10%. Once enough records are flipped, flipped records start appearing in each other’s neighbourhoods, so their neighbourhoods stop disagreeing with them. The defense gets worse exactly as the attack gets stronger. With low-confidence selection the drop is much smaller (50% to 46.5%), because those records are scattered near the boundary rather than clustered.

What it costs. The defense discarded 54 of 2000 clean records (2.7%). Test accuracy after filtering was 0.8179, slightly *above* the 0.8132 clean baseline — the filter mostly removes records that were genuinely ambiguous, so in this setting the defense cost us nothing.

6.4.4 What this defense cannot do

We do not claim it is enough.

- An attacker who knows about the neighbour check can add a term to the optimisation that keeps the poison’s neighbours label-consistent.

- Our attacker did not know about the defense, and the defender used the same frozen extractor the attacker crafted against. Both favour the defense.
- The clean records already disagree with their neighbours quite often (mean 0.359, and the top 1% reach 1.0). The poison is caught because it is extreme, not because the feature space is tidy.
- Clean-label poisons are hard by design, because their labels really are correct.

6.4.5 The end-to-end attack needs a different defense

We ran the same neighbour check in the end-to-end setting and it failed completely: AUC 0.4988, which is chance, and none of the 50 poisons flagged.

The reason is that the check asks a local question. In transfer learning the poison *has* to sit inside the target’s feature-space neighbourhood, because that is the only way one record can move a linear head, so a neighbour check finds it. In the end-to-end setting the poison does not need to sit anywhere in particular at filtering time; it only has to pull the target during retraining. There is no local inconsistency to find. (The victim also trains to a loss of 0.0007, so it has memorised its training set and every clean record scores zero disagreement — there is no distribution left to threshold.)

What the poisons do share is a direction: all 50 were crafted toward the same target, so they move together. Spectral signatures [2] looks for exactly that. Take every record labelled as the base class, extract its features from the poisoned model, subtract the class mean, and score each record by its squared projection onto the top singular vector.

	Neighbour check	Spectral signature
What it asks	do my neighbours share my label?	do I move with a group?
AUC	0.4988	0.9977

Table 12: Two defenses against the same end-to-end attack. Each one matches the mechanism of the attack it can see.

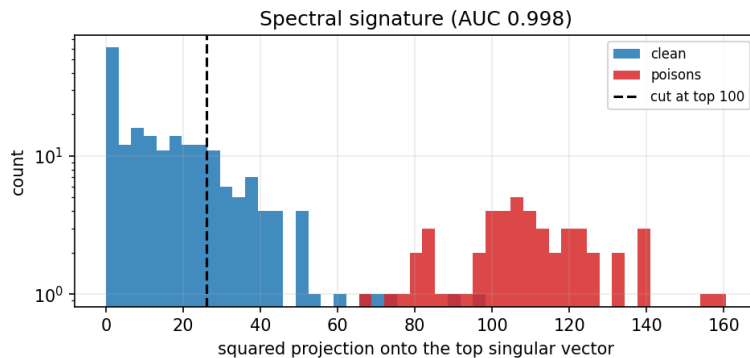


Figure 17: Squared projection onto the top singular vector, on a log scale. The poisons sit in a tail the clean records do not reach.

Detecting the poisons and stopping the attack are different questions, so we filtered and then retrained the victim on what survived.

Records removed	Poisons caught	Clean records lost	Target after retraining
none (poisoned model)	—	—	frog (attack succeeded)
top 75	46/50 (92%)	14.5%	frog (attack survives)
top 100	50/50 (100%)	25.0%	airplane (attack stopped)

Table 13: Catching most of the poisons was not enough. Test accuracy recovered from 0.7929 to 0.8152 against a clean baseline of 0.8237.

The middle row is from our random-target run, where four poisons survived the filter and the target still came out as the base class. Surviving poisons do not need numbers, because they were all crafted toward the same target and therefore pull in the same direction. Only when the filter removed every poison did the target come back.

The cost is high, and it does not fall where you would expect. Removing the top 100 discards 50 of the 200 clean frog records — a quarter of that class. Two things make this less bad than it sounds and one makes it worse.

Test accuracy after retraining was 0.8152 against a clean baseline of 0.8237, so losing a quarter of one class cost under one point overall: at 200 records per class the model has more data than it strictly needs. And the defender only pays this on the class it suspects, not across the whole dataset.

What makes it worse is that we chose to remove a fixed top 100 rather than setting a threshold, which quietly assumes the defender already knows roughly how many poisons to expect. A defender without that knowledge has to guess, and the previous row shows what happens to a guess that is slightly too low. In the transfer setting the same family of defense cost only 2.7% of clean records for a comparable catch rate, so this is a property of the end-to-end case rather than of filtering in general.

We conclude that controlling where training data comes from matters more than any single filter applied afterwards, which is also the point the authors of [3] make.

7 Reproducing our results

Everything is seeded from `SEED = 406` for the clean label task and from `SEED = 42` for the label flipping task. This fixes which images are selected, which base and targets are used, and the order the model sees the data. The pretrained weights are pinned to one specific checkpoint (`ResNet18_Weights.IMAGENET1K_V1`) rather than “the latest one”, so a rerun cannot silently pick up different weights.

Notebook	What it produces
<code>transfer_learning.ipynb</code>	Attack 2 (transfer), Attack 1, both defense layers
<code>e2e.ipynb</code>	Attack 2 (end-to-end), imbalance control, spectral signatures
<code>svm_poisoning.ipynb</code>	the optimised injection attack
<code>svm_defense.ipynb</code>	RONI, hybrid RONI and data sanitization

We wrote all attack and defense logic ourselves. PyTorch and scikit-learn are used only to fit models. No ready-made attack library was used.

8 Conclusion

The two attacks in this project differ in almost every way that matters, and the only thing they share is the channel they arrive through. The attack on the SVM only required a minuscule 1% contamination of the training set (a single optimized point) to inflict catastrophic, highly visible damage, immediately announcing itself through a massive spike in the overall error rate. Attack 2 needed one record in two thousand, left accuracy unchanged to within 0.001, and was wrong about exactly the input its author chose.

That difference decided which defenses could work. Comparing labels against a trusted source — the first thing anyone would try — caught every flipped label and not one poison, because a clean-label poison’s label is correct. Checking whether removing a record improves the model caught flips and left no trace for the poison, whose effect was smaller than ordinary retraining noise. Only looking at where a record sits relative to its neighbours in feature space saw both. And even that did not transfer: the same check scored at chance against the same attack on a victim that retrains all of its layers, where the poisons no longer have to sit anywhere in particular, and a method looking for a shared direction across the group was needed instead.

So a defense is not good or bad on its own. It answers one question, and it works exactly as long as the attack it faces has to answer that question too. Every filter we built could be evaded by an attacker who knew it was there, at some cost in attack strength.

What that leaves is the point both papers make and our results support: controlling where training data comes from is worth more than any filter applied after it arrives. A poison has to enter the training set to work, and that is the one step at which it is cheap to stop.

References

- [1] B. Biggio, B. Nelson, and P. Laskov. *Poisoning Attacks against Support Vector Machines*. ICML 2012; arXiv:1206.6389v3.
- [2] B. Tran, J. Li, and A. Madry. *Spectral Signatures in Backdoor Attacks*. NeurIPS 2018; arXiv:1811.00636.
- [3] A. Shafahi, W. R. Huang, M. Najibi, O. Suciu, C. Studer, T. Dumitras, and T. Goldstein. *Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks*. NeurIPS 2018; arXiv:1804.00792v2.